GraphQL Demo Project

Overview

This project demonstrates a basic GraphQL implementation using Java 21, Spring Boot, and Spring GraphQL.

The application provides:

- GraphQL Queries
- GraphQL Mutations
- In-memory data handling
- Layered architecture
- Error handling
- Dynamic query execution

The goal of this demo is to understand how GraphQL works compared to traditional REST APIs.

Technology Stack

Technology	Purpose
Java 21	Backend Language
Spring Boot	Application Framework
Spring GraphQL	GraphQL Integration
Maven	Build Tool
GraphQL	API Query Language

Project Architecture

```
Client (Postman / Browser)
        ↓
POST /graphql
        ↓
GraphQL Engine
        ↓
Schema Mapping
        ↓
Controller (Resolver)
        ↓
Service Layer
```

```
        ↓
Repository / In-Memory Storage
        ↓
JSON Response
```

Project Structure

```
graphql-demo
 ├ src/main/java
 │ ├ controller
 │ │ └ UserController.java
 │ ├ service
 │ │ └ UserService.java
 │ ├ repository
 │ │ └ UserRepository.java
 │ ├ dto
 │ │ └ UserDTO.java
 │ ├ entity
 │ │ └ UserEntity.java
 │ └ GraphqlDemoApplication.java
 │
 ├ src/main/resources
 │ └ graphql
 │   └ schema.graphqls
 │
 ├ pom.xml
 └ README.md
```

GraphQL Schema

File

```
src/main/resources/graphql/schema.graphqls
```

Schema Definition

```
type Query {
    users: [User]
    userById(id: ID!): User
}
```

```
type Mutation {
    createUser(name: String!, email: String!): User
}

type User {
    id: ID
    name: String
    email: String
}
```

Schema Explanation

Query

Used for fetching data.

users

Returns all users.

userById

Returns a single user based on ID.

Mutation

Used for create/update/delete operations.

createUser

Creates a new user.

Controller Layer

File

```
UserController.java
```

Responsibilities

- Maps GraphQL operations
- Receives GraphQL requests

- Calls service layer
- Returns GraphQL responses

Example

```
@QueryMapping
public List<UserDTO> users() {
    return userService.getUsers();
}
```

Important Annotations

Annotation	Purpose
@QueryMapping	Handles GraphQL Query
@MutationMapping	Handles GraphQL Mutation
@Argument	Reads GraphQL Input Parameter
@Controller	Spring Controller

Service Layer

File

```
UserService.java
```

Responsibilities

- Business logic
- Data processing
- Validation
- Repository communication

Example Methods

```
getUsers()
getUserById(String id)
createUser(String name, String email)
```

Repository Layer

File

```
UserRepository.java
```

Responsibilities

- Stores data
- Fetches data
- Simulates database operations

This demo uses in-memory storage for simplicity.

DTO Layer

File

```
UserDTO.java
```

Purpose

Used to transfer data between layers.

End-to-End Flow

Query Flow

```
Client
  ↓
/graphql
  ↓
GraphQL Schema
  ↓
@QueryMapping Method
  ↓
Service Layer
  ↓
Repository
```

```
  ↓
JSON Response
```

Example Query

Fetch All Users

```
{
  "query": "{ users { id name email } }"
}
```

Response

```
{
  "data": {
    "users": [
      {
        "id": "1",
        "name": "Rohit",
        "email": "rohit@test.com"
      }
    ]
  }
}
```

Example Mutation

Create User

```
{
  "query": "mutation { createUser(name:\"Rohit\", email:\"rohit@test.com\")
{ id name email } }"
}
```

Response

```
{
  "data": {
    "createUser": {
      "id": "1",
```

```
      "name": "Rohit",
      "email": "rohit@test.com"
    }
  }
}
```

Error Handling

Example

```
if (!email.contains("@")) {
    throw new RuntimeException("Invalid email");
}
```

GraphQL Error Response

```
{
  "errors": [
    {
      "message": "INTERNAL_ERROR"
    }
  ]
}
```

Difference Between REST and GraphQL

REST	GraphQL
Multiple Endpoints	Single Endpoint
Fixed Response	Client Chooses Fields
Over-fetching Possible	Exact Data Fetch
URL-based Routing	Schema-based Routing

Advantages of GraphQL

- Reduces over-fetching
- Single endpoint architecture
- Flexible data retrieval

- Strong schema contract
- Better frontend control

Future Enhancements

The project can be extended with:

- MySQL/PostgreSQL Integration
- Spring Data JPA
- JWT Authentication
- Kafka Event Publishing
- Redis Cache
- GraphQL Subscriptions
- Docker Deployment
- React/Angular Frontend

README

GraphQL Demo Application

Introduction

This project demonstrates a simple GraphQL implementation using Spring Boot and Java 21.

The application supports:

- Fetching all users
- Fetching user by ID
- Creating users using GraphQL mutation

Prerequisites

Install the following:

- Java 21
- Maven
- IntelliJ IDEA
- Postman

Clone Project

```
git clone <repository-url>
```

Open Project

1. Open IntelliJ IDEA
2. Select Open Project
3. Choose project folder
4. Wait for Maven dependencies to download

Run Application

Option 1

Run:

```
GraphqlDemoApplication.java
```

Option 2

Using terminal:

```
mvn spring-boot:run
```

Application URL

```
http://localhost:8080/graphql
```

Testing Using Postman

Request Type

```
POST
```

URL

```
http://localhost:8080/graphql
```

Headers

```
Content-Type: application/json
```

Fetch All Users

Request Body

```
{
  "query": "{ users { id name email } }"
}
```

Fetch User By ID

Request Body

```
{
  "query": "{ userById(id:\"1\") { id name email } }"
}
```

Create User

Request Body

```
{
  "query": "mutation { createUser(name:\"Rohit\", email:\"rohit@test.com\")
{ id name email } }"
}
```

Common Issues

404 Error

If you open:

```
http://localhost:8080/
```

You may see:

```
Whitelabel Error Page
```

This is expected because GraphQL endpoint is:

```
/graphql
```

Important Notes

- GraphQL typically uses POST requests
- All operations go through a single endpoint
- Schema controls available operations
- Controller methods map directly to schema names

Author Notes

This project was created as a learning/demo application to understand GraphQL architecture and Spring GraphQL integration.

Conclusion

This demo provides a clean introduction to GraphQL with Spring Boot and demonstrates:

- GraphQL schema design
- Query and mutation handling
- Layered architecture
- Spring GraphQL integration
- Dynamic data fetching